Homework 08

The former kernel-tuning question has moved to Homework 09.

Solutions for Questions 1 and 2 will be available after Monday, October 19, 2026. No solution can be provided for Question 3.

For Questions 1 and 2, use either R or Python.

Question 1: Boundary and margin

Use `margin-geometry-20.csv` from the combined ZIP, which contains 20 observations with predictors x_1, x_2 and class labels $y \in \{-1, 1\}$. Consider the score

$$f(x) = 0.8x_1 + 0.6x_2.$$

Since $\|(0.8, 0.6)\|_2 = 1$, the lines $f(x) = \pm 1$ are each a distance 1 from the decision boundary $f(x) = 0$.

a. Calculate the distance from every observation to the decision boundary and its functional margin $yf(x)$. In a table, report these values and whether each observation is inside, on, or beyond the margin for its class. Compare $yf(x)$ with 1 to make this distinction.

b. Plot the observations, the boundary $f(x) = 0$, and the margin lines $f(x) = \pm 1$. Distinguish the two classes and identify the observations inside the margin in your plot.

c. Report how many observations are inside, on, and beyond the margin. Which observations are misclassified by this decision rule?

Use the observation numbers supplied in the data. Set the x_1 axis to $[-3.2, 3.2]$, set the x_2 axis to $[-3, 3]$, and use an equal aspect ratio. Do not fit a new classifier for this question.

Answer (a)

Since $\|\beta\|_2 = 1$, the distance to the decision boundary is $|f(x_i)|$, and the functional margin is $y_i f(x_i)$.

```
d <- read.csv("data/margin-geometry-20.csv")

f <- 0.8*d$x1 + 0.6*d$x2
m <- d$y * f

result <- data.frame(
  Observation = d$observation,
  Distance = abs(f),
  Functional_Margin = m,
  Position = ifelse(abs(m - 1) < 1e-8, "On",
                    ifelse(m < 1, "Inside", "Beyond"))
)

knitr::kable(result, digits = 4)
```

Observation	Distance	Functional_Margin	Position
1	1.00	1.00	On
2	1.35	1.35	Beyond
3	1.50	1.50	Beyond
4	1.70	1.70	Beyond
5	1.90	1.90	Beyond
6	2.20	2.20	Beyond
7	2.40	2.40	Beyond
8	0.64	0.64	Inside
9	0.20	0.20	Inside
10	0.20	-0.20	Inside
11	1.00	1.00	On
12	1.30	1.30	Beyond
13	1.55	1.55	Beyond
14	1.75	1.75	Beyond
15	1.95	1.95	Beyond
16	2.15	2.15	Beyond
17	2.45	2.45	Beyond
18	0.64	0.64	Inside
19	0.24	0.24	Inside
20	0.20	-0.20	Inside

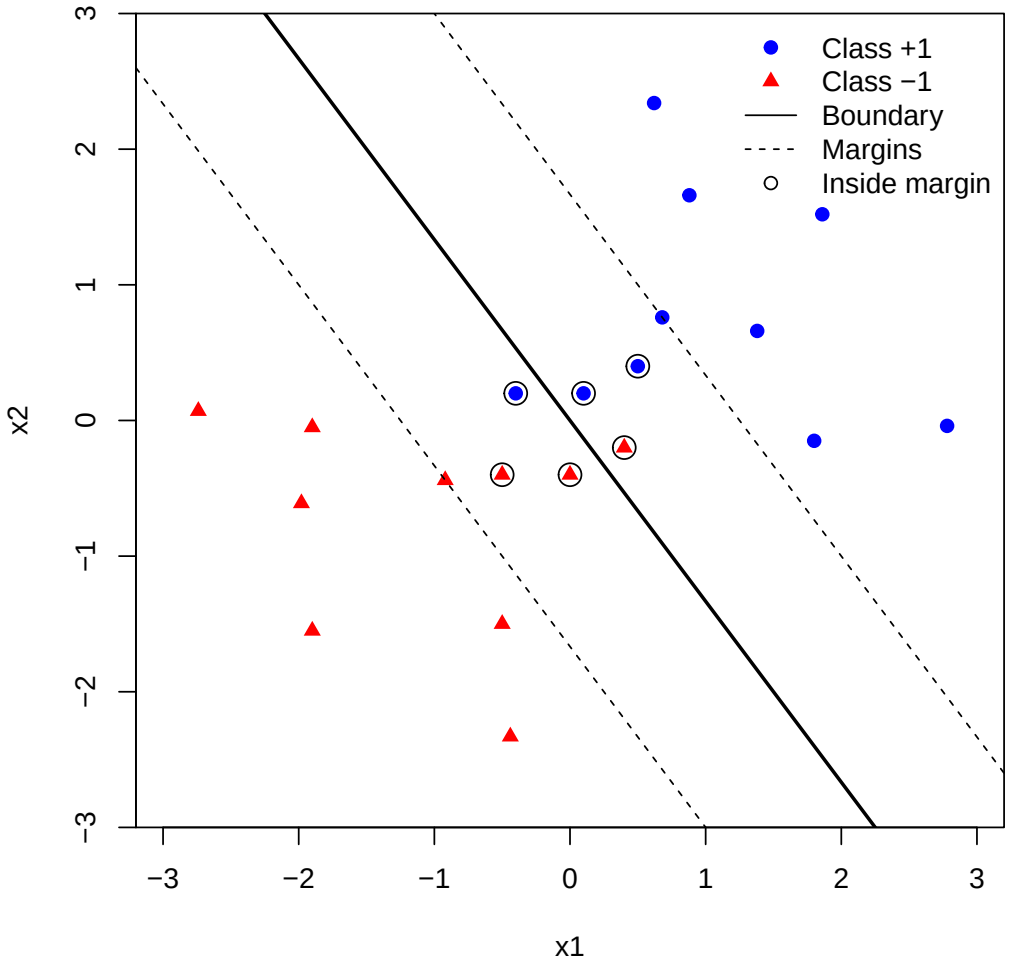
Answer (b)

```
par(pin = c(5.12, 4.8))
plot(d$x1, d$x2,
     col = ifelse(d$y == 1, "blue", "red"),
     pch = ifelse(d$y == 1, 19, 17),
     xlim = c(-3.2, 3.2), ylim = c(-3, 3),
     asp = 1, xaxs = "i", yaxs = "i", xlab = "x1", ylab = "x2")

abline(a = 0, b = -4/3, lwd = 2)
abline(a = 5/3, b = -4/3, lty = 2)
abline(a = -5/3, b = -4/3, lty = 2)

inside <- result$Position == "Inside"
points(d$x1[inside], d$x2[inside], pch = 1, cex = 1.8)

legend("topright",
       legend = c("Class +1", "Class -1", "Boundary",
                  "Margins", "Inside margin"),
       col = c("blue", "red", "black", "black", "black"),
       pch = c(19, 17, NA, NA, 1),
       lty = c(NA, NA, 1, 2, NA), bty = "n")
```



Answer (c)

```
table(result$Position)
```

```
Beyond Inside     On
    12      6      2
```

```
d$observation[m < 0]
```

```
[1] 10 20
```

There are 6 observations inside the margin, 2 on the margin, and 12 beyond the margin. Observations 10 and 20 are misclassified because their functional margins $y_i f(x_i)$ are negative (both equal to -0.2).

Question 2: Margin in four dimensions

Use `margin-geometry-4d.csv` from the combined ZIP, which contains 50 observations with four predictors and class labels $y \in \{-1, 1\}$. Consider the linear score

$$f(\mathbf{x}) = \beta_0 + \mathbf{x}^\mathsf{T}\beta, \qquad \beta_0 = -1, \quad \beta = (1, 2, -2, 4)^\mathsf{T}.$$

a. Project the observations onto the direction of β and plot their positions on one axis. Distinguish the two classes and mark the decision boundary. Use the plot to calculate the geometric margin M, the distance from the boundary to the closest observation.

b. Express the same decision boundary using the SVM normalization in which the smallest functional margin $y_i f(\mathbf{x}_i)$ is set to 1. What are the corresponding values of β and β_0?

Answer (a): Geometric margin

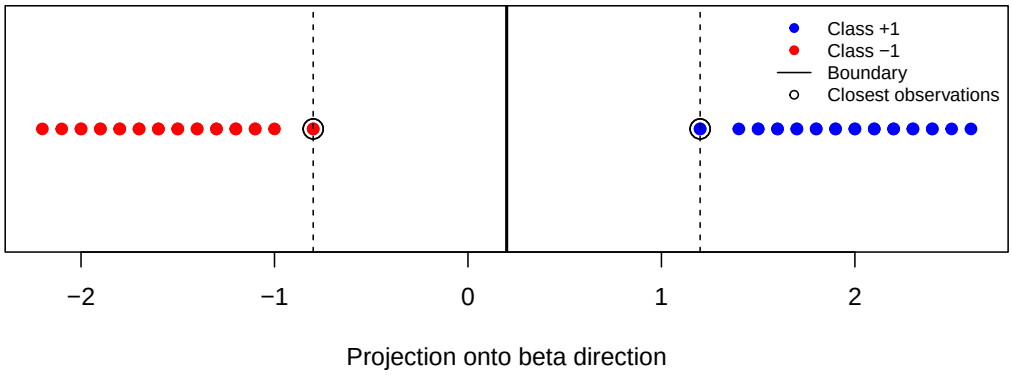
The coefficient vector is $\beta = (1, 2, -2, 4)^\mathsf{T}$, with $\|\beta\|_2 = 5$. Project each observation onto its unit direction:

$$t_i = \frac{\mathbf{x}_i^\mathsf{T}\beta}{5}.$$

The decision boundary is at $t = -\beta_0/5 = 0.2$.

```
d4 <- read.csv("data/margin-geometry-4d.csv")
beta <- c(1, 2, -2, 4)
X4 <- as.matrix(d4[, c("x1", "x2", "x3", "x4")])
projection <- as.vector(X4 %*% beta) / sqrt(sum(beta^2))
boundary <- 1/5
M <- min(abs(projection - boundary))
closest <- abs(abs(projection - boundary) - M) < 1e-8
```

```r
plot(projection, rep(0, length(projection)),
     col = ifelse(d4$y == 1, "blue", "red"),
     pch = 19, yaxt = "n", ylab = "", ylim = c(-0.5, 0.5),
     xlab = "Projection onto beta direction")
abline(v = boundary, lwd = 2)
abline(v = boundary + c(-M, M), lty = 2)
points(projection[closest], rep(0, sum(closest)), pch = 1, cex = 1.8)
legend("topright",
       legend = c("Class +1", "Class -1", "Boundary", "Closest observations"),
       col = c("blue", "red", "black", "black"),
       pch = c(19, 19, NA, 1), lty = c(NA, NA, 1, NA),
       bty = "n", cex = 0.75)
```




```r
M
```

```
[1] 1
```

The closest projected observations are at $t = -0.8$ and $t = 1.2$, each a distance 1 from the boundary at $t = 0.2$. Thus,

$$\boxed{M = \min_i |t_i - 0.2| = 1}.$$

Answer (b): SVM normalization

```
f4 <- -1 + as.vector(X4 %*% beta)
min_margin <- min(d4$y * f4)
min_margin
```

```
[1] 5
```

```
beta_star <- beta / min_margin
beta0_star <- -1 / min_margin
beta_star
```

```
[1]  0.2  0.4 -0.4  0.8
```

```
beta0_star
```

```
[1] -0.2
```

```
min(d4$y * (beta0_star + as.vector(X4 %*% beta_star)))
```

```
[1] 1
```

The minimum functional margin is $\min_i y_i f(\mathbf{x}_i) = 5$. Dividing all coefficients by 5 preserves the decision boundary and sets the smallest functional margin to 1:

$$\boxed{\beta^* = (0.2, 0.4, -0.4, 0.8)^\top}, \qquad \boxed{\beta_0^* = -0.2}.$$

The normalized decision boundary is

$$-0.2 + 0.2x_1 + 0.4x_2 - 0.4x_3 + 0.8x_4 = 0.$$

Question 3: Peer review of a Homework 7 skill

The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 7 submission. It will link to **one** assigned student's repository.

1. **Use the skill.** Find the complete skill definition in the assigned student's Homework 7 report. Copy it without changes into a local `SKILL.md` file in your own folder. You do not need to install it. Give your agent the path to the file and explicitly ask it to follow that skill while analyzing the `SAheart` data, using the response and predictor coding from Homework 7. Provide the necessary data information, but no additional preferences about methods, figures, model selection, or report style. Let the skill direct the analysis.

   Monitor the time, your token use, and your weekly quota while the agent works. If the run is taking too long or using your allowance too quickly, you may send a **steer message** asking the agent to finish immediately, or stop the run and summarize what you observed. A steer message redirects an agent while it is working; learn how to send one in your AI tool before starting.

2. **Write a peer review.** Read the student's submitted report and what the skill produced in your run. Give a brief overall assessment, followed by **three major items the author could improve**. You may comment on any aspect of the skill or report. For each item, explain what you observed, why it matters, and what you recommend changing. If you stopped the run early, explain where and why. Write the review yourself, without AI assistance.

3. **Share your review.** Open a GitHub Issue in the assigned student's Homework 7 repository containing your review. Copy the same review into your Homework 8 submission report and include a link to the GitHub Issue.

Answer

GitHub Issue: Peer review

Overall Assessment

The overall performance is good. In the original report, the PCA-LDA model was selected, yielding a test error rate of 0.252. In my test run, standard LDA was chosen, resulting in a test error rate of 0.324. This Skill successfully guides the AI through LDA/QDA comparison, cross-validation, model selection, and test set evaluation.

Improvement 1: Clarify rules for resolving ties in model selection

In instruction 5 of the submitted Skill, the author requires "breaking ties in favor of the simpler model." However, the Skill does not define how complexity should be compared among ordinary LDA, QDA, PCA-based models, and probability blends. It also does not specify whether ties mean exactly equal cross-validation errors or errors within a numerical tolerance.

This could lead to inconsistent selections when cross-validation errors are tied. No tie occurred in my run, so this is a potential ambiguity rather than an observed selection error.

I recommend explicitly defining how to compare the complexity of tied candidates, including PCA-based and blended models. The Skill should also clarify whether ties require exactly equal errors or allow a specified numerical tolerance.

Improvement 2: Clarify how to handle model failures during cross-validation

The Skill requirements mandate logging failures, yet there are no specific rules regarding how to handle cases where a model fails on only a subset of folds. This creates potential ambiguity in execution.

As an improvement, I believe that rules for re-running the model after a failure should be established in advance. If the model still fails to complete all folds, it should be excluded from the pool of comparable candidate models, and the reason for the exclusion should be logged.

Improvement 3: Preserve execution details for reproducibility

The third point concerns reproducibility and the documentation of the execution process as well. Currently, the Skill provides code only upon request. To facilitate the reproduction of the execution process by others, the Skill could be configured to save—by default—the code, data splits, cross-validation fold indices, candidate parameter grids, and software versions. This does not imply saving vast amounts of unnecessary code by default; rather, it means preserving sufficient information to enable others to accurately reproduce the model selection and evaluation processes.